

IoTSuite: A ToolSuite for Prototyping Internet of Things Applications

Dimitris Soukaras*, Pankesh Patel†, Hui Song‡, and Sanjay Chaudhary§

*University of Peloponnese, Greece

dsoukaras@gmail.com

†ABB Corporate Research, India

pankesh.patel@in.abb.com

‡SINTEF ICT, Oslo, Norway

hui.song@sintef.no

§Institute of Engineering & Technology, Ahmedabad University, India

sanjay.chaudhary@ahduni.edu.in

Abstract—Internet of Things (IoT) applications will involve interactions among extremely large numbers of heterogeneous devices, many of them directly interacting with their physical surroundings. An important challenge that needs to be addressed is to enable the rapid development of IoT applications with minimal effort by the various stakeholders involved in the process. However, IoT application development is challenging because it involves dealing with issues that are attributed to different life-cycle phases, including development, deployment, and evolution.

This paper presents a design and implementation of IoTSuite, a suite of tools for IoT applications development, for reducing development effort. It consists of the following components to aid stakeholders: (1) *an editor* supports application design phase by allowing stakeholders to specify high-level specifications, (2) *a compiler* parses the high-level specification and supports the application development phase by producing programming framework that reduces development effort in specifying the details of components of an IoT application, (3) *a deployment module* is supported by mapper and linker modules. They together support the deployment phase by producing device-specific code to result in a distributed system collaboratively hosted by individual devices, and (4) *a runtime system*, which leverages existing middleware platforms and responsible for a distributed execution of an IoT application. The current implementation of IoTSuite targets both Android and JavaSE-enabled devices and MQTT middleware.

I. INTRODUCTION

The upcoming field of IoT [1] will involve interactions among extremely large numbers of disparate devices, many of them directly interacting with their physical surroundings. An important challenge that needs to be addressed is to enable the rapid development of IoT applications with minimal effort by the various stakeholders¹ involved in the process. Similar challenges have already been addressed in the closely related fields of Wireless Sensor Networks and Ubiquitous/Pervasive computing, regarded as precursors to the modern day IoT. While the main challenge in the former is the *large scale* – hundreds to thousands of largely similar devices, the primary concern in the latter has been the *heterogeneity* of devices and

the major role that the user’s own interaction with these devices plays in these systems (cf. the classic “smart home” scenario where a user controls lights and receives notifications from his refrigerator and toaster.). It is the goal of our work to enable the development of such applications.

A simple example of an IoT application can be found in building automation domain. A building system might consist of several buildings, with each building in turn consisting of one or more floors, each with several rooms. It may consist of a large number of heterogeneous devices equipped with sensors (e.g., temperature sensor, smoke detector), actuators (e.g., alarm, heater), and storage (e.g., profiledb to store user’s profile) to store the persistent information. An application, installed in this building system, system generates the current status (e.g., temperature, energy consumption) of each room, which is then aggregated and used to determine the current status of each floor and, in turn, the entire building. A monitor installed at the building entrance presents the information to the building operator for situational awareness. It can thus encourage residents to save energy.

Application development, as mentioned above, is challenging because stakeholders have to address issues that are attributed to different life cycles phases, including development, deployment, and evolution. At the *development phase*, the application logic has to be analyzed and separated into a set of distributed tasks for the underlying network consisting of a large number of heterogeneous entities. Then, the tasks have to be implemented for the specific platform of a device. At the *deployment phase*, the application logic has to be deployed onto a large number of devices. Apart from handling these issues, stakeholders have to keep in mind evolution issues both in the development (e.g., change in functionality of an application) and deployment phase (e.g. adding, removing devices in deployment scenarios) at the *evolution phase*. Manual effort in all above three phases for hundreds to thousands of heterogeneous devices is a time-consuming and error-prone process. Thus, an IoT application development requires tool support during all phases of the development.

Contributions. In our previous work [2], [3], [4], we have proposed a development framework. Since the main goal

¹we use the term stakeholders as used in software engineering to mean people, who are involved in the application development. Examples of stakeholders are software designer, developer, domain expert, technologist, etc.

of our research is to reduce development effort² for stakeholders, we believe that our development framework should be supported by tools to be applicable in an effective way. This paper presents an implementation of the development framework, realized as a suite of tools. We call it as IoTSuite³. Specifically, it consists of the following components to aid stakeholders: *an editor* supports application design phase by allowing stakeholders to specify high-level specifications, *a compiler* parses the high-level specification and supports the application development phase by producing programming framework that reduces development effort in specifying the details of components of an IoT application, *a deployment module* is supported by mapper and linker modules. They together support the deployment phase by producing device-specific code to result in a distributed system collaboratively hosted by individual devices, and *a runtime system*, which leverages existing middleware platforms and responsible for a distributed execution of an IoT application.

Outline. The remainder of this paper is organized as follows: Section II summarizes our IoT application development process. The next section describes different components that stakeholders can use at each stage of development process. Section IV presents existing work available for IoT application development. Finally, Section V summarizes this paper and discusses one of our future directions.

II. OUR APPROACH

To provide the reader necessary background, this section summarizes our IoT application development process (complete tour is available in our previous publications [2], [3], [4]), illustrated in Figure 1. It separates IoT application development into different concerns and integrates a set of high-level languages to specify them. It is supported by compiler, mapper, and linker modules at various phases of IoT application development process to provide automation. Stakeholders carry out the following steps in order to develop an IoT application using our approach.

Specifying domain vocabulary. The domain expert specifies a domain vocabulary (step ① in Figure 1). The vocabulary specification includes concepts specific to a target application domain (e.g., building automation, transport, etc.). For example, the building automation domain is reasoned in terms of rooms and floors, while the transport domain is expressed in terms of highway sectors. Furthermore, the vocabulary includes specification of resources, which are responsible for interacting with entities of interest (EoI). This includes sensors (sense EoI), actuators (control EoI), and storage (store information about EoI).

Compiling vocabulary specification. Leveraging the vocabulary, the toolsuite generates (step ② in Figure 1): (1) a vocabulary framework to aid the device developer, (2) a customized architecture grammar according to the vocabulary to aid the software designer, and (3) a customized deployment grammar according to the vocabulary to aid the network manager. The

key advantage of this customization is that domain-specific concepts defined in the vocabulary are made available to other stakeholders and can be reused across applications of the same application domain.

Specifying application architecture. Using a customized architecture grammar, the software designer specifies an application architecture (step ③ in Figure 1). He specifies computational services and interactions with other components. Computational services are fueled by sensors and storage (defined in the vocabulary). They process inputs data and take appropriate decisions by triggering actuators (defined in the vocabulary specification).

Compiling architecture specification. The development framework leverages an architecture specification to support the application developer (step ④ in Figure 1). To describe the application logic of each computational service, the application developer is provided an architecture framework, pre-configured according to the architecture specification of an application, an approach similar to the one discussed in [6].

Implementing application logic. To describe the application logic of each computational service, the application developer leverages a generated architecture framework (step ⑤ in Figure 1). It contains abstract classes⁴, corresponding to each computational service, that hide interaction details with other software components and allow the application developer to focus only on application logic. The application developer implements only the abstract methods of generated abstract classes.

Specifying target deployment. Using a customized deployment grammar, the network manager describes a deployment specification (step ⑥ in Figure 1). The deployment specification includes the details of each device, including its regions (in terms of values of the regions defined in the vocabulary), resources hosted by devices (a subset of those defined in the vocabulary), and the type of the device. Ideally, the same IoT application could be deployed on different target deployments (e.g., the same inventory tracking application can be deployed in different warehouses). This requirement is dictated by separating a deployment specification from other specifications.

Mapping. The mapper produces a mapping from a set of computational services to a set of devices (step ⑦ in Figure 1). It takes as input a set of placement rules of computational services from an architecture specification and a set of devices defined in a deployment specification. The mapper decides where each computational service will be deployed. The current version of algorithm [5] selects devices randomly and allocates computational services to the selected devices. A mapping algorithm aware of heterogeneity, associated with devices of a target deployment, is a part of our future work.

Implementing device drivers. Leveraging the vocabulary, IoTSuite generate a vocabulary framework to aid the device developer (step ⑧ in Figure 1). The vocabulary framework contains a set of *interfaces* and *concrete classes* corresponding to resources defined in the vocabulary. The concrete classes

²We have used the lines of code metric for evaluating development effort in our work [5].

³An open source version, targeting on Android- and JavaSE -enabled devices and MQTT middleware, is available on: <https://github.com/pankeshlinux/IoTSuite/wiki>

⁴We assume that the application developer uses an object-oriented language.

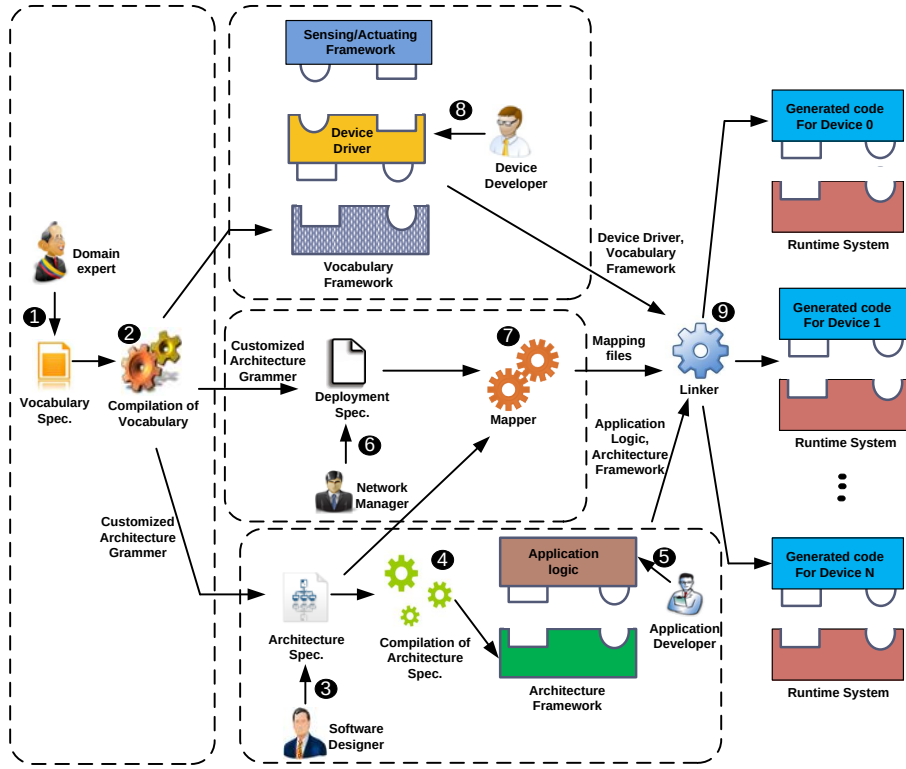


Fig. 1: IoT application development: overall process

contain concrete methods for interacting with other software components and platform-specific device drivers. We have integrated existing open-source sensing frameworks⁵ for Android devices. So, the device developer has to only implement interfaces, connecting integrated sensing framework and generated vocabulary framework.

Linking. The linker combines and packs code generated by various stages into packages that can be deployed on devices (step ⑨ in Figure 1). This stage supports the application deployment phase by producing device-specific code to result in a distributed software system collaboratively hosted by individual devices, thus providing automation at the deployment phase⁶.

Evolution. Evolution is an important aspect in IoT application development where sensors, actuators, and computational services are added, removed, or extended. To deal with these changes, we separate IoT application development into different concerns and allow an iterative development for these concerns. This iterative development requires only a change in evolved specification and reusing dependent specifications/implementation in compilation process, thus reducing effort to handle evolution, similar to the work in [6].

III. COMPONENTS OF IOTSUITE

In the following, we demonstrate the above mentioned application development using IoTSuite, composed of different

components, mentioned below, at each phase of application development that stakeholders can use.

Editor. It helps stakeholders to write high-level specifications, including vocabulary, architecture, and deployment specification with the facilities of syntax coloring and syntax error reporting. We use Xtext⁷ for a full fledged editor support, similar to work in [7]. The Xtext is a framework for a development of domain-specific languages, and provides an editor with syntax coloring by writing Xtext grammar.

We take an example of building automation domain vocabulary to demonstrate an editor support provided by IoTSuite, illustrated in Figure 2. The zone ① shows the editor, where the domain expert writes a domain vocabulary. The zone ② shows the menu bar, where the domain expert invokes the compiler for vocabulary to generate a framework, a customized architecture grammar, and deployment grammar.

Compiler. The compiler parses high-level specifications and translates them into code that can be used by other components in the system. The parser module of compiler is implemented using ANTLR⁸, a well-known parser generator that creates parser files from grammar descriptions. The code generator module of compiler manages a repository of plug-ins. Each plug-in, defined as template files, is specific to a target implementation language (e.g., Java, Python). The key advantage of it is that it simplifies an implementation of a new code generator for a target implementation. The plug-ins are

⁵<http://www.funf.org/>

⁶We assume that a middleware is already installed on the deployed devices. The installed middleware enables inter-device communication among devices.

⁷<http://www.eclipse.org/Xtext/>

⁸<http://www.antlr.org/>

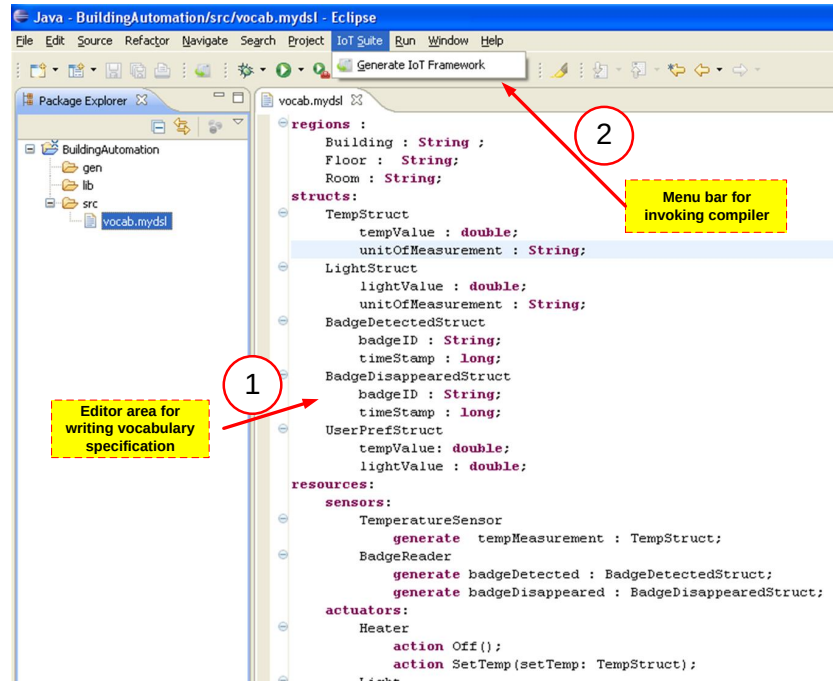


Fig. 2: Editor support for writing vocabulary specification in IoT Suite.

implemented using StringTemplate Engine,⁹ a Java template engine for generating source code or any other formatted text output. We build two compilers to aid stakeholders shown in Figure 1. (1) compiler for a vocabulary specification, and (2) compiler for an architecture specification. The current version of these compilers generate frameworks, compatible with Eclipse IDE.

Mapper. The mapper produces a mapping from a set of computational services to a set of devices. Figure 3 illustrates the architecture of the mapper component. The parser converts high-level specifications into appropriate data structures that can be used by a mapping algorithm. The mapping algorithm produces mapping decisions into appropriate data structures. The code generator consumes the data structures and generates mapping files. Our current implementation of the mapper randomly maps computational services to a set of devices. However, due to generality of our architecture, more sophisticated mapping algorithm can be plugged into the mapper.

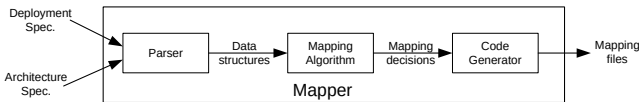


Fig. 3: Architecture of the mapper component in IoT Suite.

Linker. It combines and packs code generated by various stages of compilation into packages that can be deployed on devices. The output of the linker is a set of platform-specific projects for devices, specified in the deployment specification. These projects are not binaries. They need to be compiled, which can be done by any device-specific compiler designed

for the target platform. The current version of the linker generates Java source packages for Android and JavaSE platform. Figure 4 illustrates packages for 3 target devices (2 packages for JavaSE devices and 1 for Android device), which can be imported into Eclipse IDE.

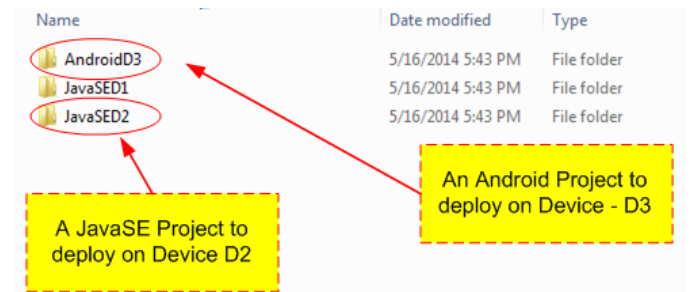


Fig. 4: Packages for target devices compatible with Eclipse IDE

Runtime system. The main responsibility of the runtime system is a distributed execution of IoT applications. It is composed of three parts: (1) *middleware*: It runs on each individual device and provides a support for executing distributed tasks. (2) *wrapper*: It plugs packages, generated by the linker module, and middleware. (3) *support library*: It separates packages, produced by the linker component, and underlying middleware by providing interfaces that are implemented by each wrapper. The integration of a new middleware into IoT Suite consists of an implementation of the following interfaces, specified by the support library, in the wrapper.

- `publish()`. It is an interface for publishing data from a sender. The definition of this interface contains: an event name (e.g., temperature), event data

⁹<http://www.stringtemplate.org/>

(e.g., a temperature value, Celsius), and publisher's information such as location.

- `subscribe()`. It is an interface for receiving event notifications. An interest of events is expressed by sending a subscription request, which contains: an event name (e.g., temperature), information for filtering events such as regions of interest (e.g., a RoomAvgTemp component wants to receive events only from a current room), and subscriber's information.
- `command()`. It is an interface for triggering an action of an actuator. A command contains: a command name (e.g., switch-on heater), command parameters (e.g., set temperature of heater to 30°C), and a sender's information.
- `request-response()`. It is an interface for requesting data from a requester. In reply, a receiver sends a response. A request contains a request name (e.g., give profile information), request parameters (e.g., give profile of person with identification 12), and information about the requester.

The current implementation of IoTSuite uses the MQTT¹⁰ middleware, which enables interactions among Android devices and JavaSE enabled devices.

IV. RELATED WORK

This section presents some of existing works for IoT application development. In the following, we briefly present each approach. Furthermore, Table I summarizes implementation technologies, tools, language used in each approach.

DiaSuite [6] is a suite of tools to develop pervasive computing applications. It combines design languages and covers application development life-cycle. Stakeholders define entities in a high-level manner to abstract heterogeneity using the design languages. However, the consideration of a large number of devices in an application is largely missing. Moreover, the application deployment for a large number of heterogeneous devices using this approach is difficult because stakeholders require manual effort (e.g., mapping of computational services to devices).

PervML [8] raises the level of abstraction in program specification, and code generators produce code from these specifications. It integrates the mapping process at the deployment phase. However, stakeholders have to link the application code and configure device drivers manually. This manual work in the deployment phase is not suitable for IoT applications involving a large number of devices. Moreover, the separation between deployment and domain-specific features are missing. These limitations would restrict PervML to a certain level.

ATaG [9] is a WSN is a macroprogramming framework to develop Sense-Compute-Control (SCC) applications. ATaG presents a compilation framework that translates a program, containing abstract notations, into executable node-level programs. Moreover, it tackles the issue of scale reasonably well. The ATaG linker and mapper modules support the application

deployment phase by producing device-specific code to result in a distributed software system collaboratively hosted by individual devices, thus providing automation at deployment phase. Nevertheless, the clear separation of roles among the various stakeholders, as well as the focus on heterogeneity among the constituent devices are largely missing.

RuleCaster [10] introduces an engineering method to provide support for SCC applications, as well as evolutionary changes in the application development. The RuleCaster compiler takes as input the application program, containing conditions and actions, and a network model that describes device locations and its capabilities. It supports automation at the deployment phase by mapping computational components to devices. However, the clear separation of roles among the various stakeholders, support for application domain, as well as the focus on heterogeneity among the constituent devices are missing.

The **Context toolkit** [11] and **VisualRDK** [12] simplify the context-aware application development on top of heterogeneous data sources. The Context toolkit provides three architectural components, namely, widgets, interpreters, and aggregators. These components separate application semantics from platform-specific code. It means stakeholders can treat a widget in a similar fashion and do not have to deal with differences among platform-specific code. Although context toolkit provides support for acquiring the context data from the heterogeneous sensors, it does not support actuation that is an essential part of IoT.

Pantagruel [13] is a visual approach dedicated to the development of home automation applications. The Pantagruel application development consists of three steps: (1) specification of taxonomy to define entities of the home automation domain (e.g., temperature sensor, alarm, door, smoke detector, etc.), (2) specification of rules to orchestrate these entities using the Pantagruel visual language, and (3) compilation of the taxonomy and orchestration rules to generate a programming framework. While this system reduces the requirement of having domain-specific knowledge for other stakeholders, the clear separation of different development concerns, support for large scale, automation at deployment phase are largely missing. These limitations make it difficult to use for IoT application development.

Physical-Virtual mashup. As indicated by its name, it connects web services from both the physical and virtual world through visual constructs directly from web browsers. The embedded device runs tiny web servers to answer HTTP queries from users for checking or changing the state of a device. For instance, users may want to see temperature of different places on map. Under such requirements, stakeholders can use the mashup to connect physical services such as temperature sensors and virtual services such as Google map. Many mashup prototypes have been developed that include both the physical and virtual services [14], [15], [16]. The mashup editor usually provides visual components representing web service and operations (such as add, filter) that stakeholders need to connect together to program an application. The framework takes care of resolving these visual components based on properties specified by stakeholders and produces code to interface software components and distributed system. The main advantage of

¹⁰<http://mqtt.org/>

this mashup approach is that any service, either physical or virtual, can be mashed-up if they follow the standards (e.g., REST). The Physical-Virtual mashup significantly lowers the barrier of the application development. However, stakeholders have to manage a potentially large graph for an application involving a large number of entities.

Existing work	Hardware platform support	Impl. language	Open Source	Installation	Runtime system support
IoTSuite	Android, J2SE	Any Object-Oriented	Yes	commandLine	Yes
DiaSuite	J2SE	Any Object-Oriented	No	Eclipse plug-in	Yes
ATaG	SunSpot, J2SE	Java	Yes	Eclipse plug-in	No
PervML	J2SE	Java, C++	No	Eclipse plug-in	No
RuleCaster	ContikiOS	C	No	Not clear	No
VisualRDK	nodes with .Net	C#	No	Not clear	No
WoTKit	Not clear	Python	No	Not clear	No
Context Toolkit	J2SE	Java	Yes	jar file	No
Pantagruel	J2SE	Any Object-Oriented	No	Not clear	Yes

TABLE I: Existing toolsuites for IoT app. dev.

V. SUMMARY AND FUTURE WORK

Since the main goal of this research is to make IoT application development easy for stakeholders, we believe that our IoT application development process should be supported by tools to be applicable in an effective way. Therefore, this paper presents a design and implementation of IoTSuite, a suite of tools, for reducing burden of each phase of IoT application development process. It integrates: (1) *editors* for writing high-level textual specifications, (2) a *compiler* to parse high-level specifications and generate a programming framework in a target implementation language to reduce development effort, (3) a *deployment module* contains mapping and linking modules, which together support the deployment phase by producing device-specific code, and (4) a *runtime system* leverages existing middleware platforms and generates glue code to customize them with respect to needs of applications. This mechanism increases the possibility of executing applications on different middleware platforms.

In our work so far, we have made initial progress towards providing support to all the stakeholders in the IoT application development process, and prepared a foundation for our future work, which will proceed in two complementary directions: (1) *Extending evaluation results*: We have used the lines of code metric for evaluating development effort in work [5]. Our near term future work will be to validate out toolkit via experimental user study as mentioned in work [8]. (2) *Supporting testing phase*: A key advantage of testing is it emulates

an application execution before deployment so as to identify possible conflicts, thus reducing application debugging effort. Currently, we are investigating Siafu¹¹ due to its open-source and thorough documentation.

ACKNOWLEDGMENT

The authors would gratefully like to thank Prof. Umesh Bellur, Dr. Pratik Desai, Dr. Balamuralidhar P, Prof. Amit Sheth, and the reviewers for their helpful comments and suggestions.

REFERENCES

- [1] J. Vasseur and A. Dunkels, *Interconnecting smart objects with IP: The next internet*. San Francisco, CA, USA: Morgan Kaufmann, 2010.
- [2] P. Patel, A. Pathak, D. Cassou, and V. Issarny, "Enabling High-Level Application Development in the Internet of Things," in *4th International Conference on Sensor Systems and Software*, Lucca, Italy, Apr. 2013.
- [3] P. Patel, A. Pathak, T. Teixeira, and V. Issarny, "Towards application development for the Internet of Things," in *Proceedings of the 8th Middleware Doctoral Symposium*. ACM, 2011, p. 5.
- [4] P. Patel, B. Morin, and S. Chaudhary, "A model-driven development framework for developing sense-compute-control applications," in *Proceedings of the 1st International Workshop on Modern Software Engineering Methods for Industrial Automation*, ser. MoSEMInA. ACM, 2014, pp. 52–61.
- [5] P. Patel, "Enabling high-level application development for the internet of things," THESE, Université Pierre et Marie Curie - Paris VI, Nov. 2013. [Online]. Available: <http://tel.archives-ouvertes.fr/tel-00927150/PDF/these.pdf>
- [6] D. Cassou, J. Bruneau, C. Consel, and E. Baland, "Towards a Tool-based Development Methodology for Pervasive Computing Applications," *IEEE Transactions on Software Engineering*, no. 99, 2011.
- [7] B. Bertran, J. Bruneau, D. Cassou, N. Lorient, E. Baland, and C. Consel, "DiaSuite: a tool suite to develop sense/compute/control applications," *Science of Computer Programming*, 2012.
- [8] E. Serral, P. Valderas, and V. Pelechano, "Towards the model driven development of context-aware pervasive systems," *Pervasive and Mobile Computing*, vol. 6, no. 2, pp. 254–280, 2010.
- [9] A. Pathak and V. Prasanna, "High-level application development for sensor networks: Data-driven approach," *Theoretical Aspects of Distributed Computing in Sensor Networks*, pp. 865–891, 2011.
- [10] U. Bischoff and G. Kortuem, "Life cycle support for sensor network applications," in *Proceedings of the 2nd international workshop on Middleware for sensor networks*. ACM, 2007, pp. 1–6.
- [11] A. Dey, G. Abowd, and D. Salber, "A conceptual framework and a toolkit for supporting the rapid prototyping of context-aware applications," *Human-Computer Interaction*, vol. 16, pp. 97–166, 2001.
- [12] T. Weis, M. Knoll, A. Ulbrich, G. Muhl, and A. Brandle, "Rapid prototyping for pervasive applications," *Pervasive Computing, IEEE*, vol. 6, no. 2, pp. 76–84, 2007.
- [13] Z. Drey, J. Mercadal, and C. Consel, "A taxonomy-driven approach to visually prototyping pervasive computing applications," in *Domain-Specific Languages*. Springer, 2009, pp. 78–99.
- [14] M. Blackstock and R. Lea, "WoTKit: a lightweight toolkit for the web of things," in *Proceedings of the Third International Workshop on the Web of Things*. ACM, 2012, p. 3.
- [15] D. Guinard, V. Trifa, and E. Wilde, "A resource oriented architecture for the web of things," in *Internet of Things (IOT)*. IEEE, 2010, pp. 1–8.
- [16] A. P. Castellani, M. Dissegna, N. Bui, and M. Zorzi, "WebIoT: A web application framework for the internet of things," in *Wireless Communications and Networking Conference Workshops (WCNCW)*. IEEE, 2012, pp. 202–207.

¹¹<http://siafusimulator.org/>